

```

return f"""<html>
    <body>
        <center>
            <h2>Welcome</h2>
            <hr>
            <a href="{base_url}/generate">Get
token</a><br><br>
            <a href="{base_url}/validate">Show
details</a>
            <p>Result - {result}</p>
        </center>
    </body>
</html>"""

@app.route('/generate')
def generate_token():
    try:
        payload = {
            'sub': 'bob',
            'iat': datetime.datetime.utcnow(),
            'exp': datetime.datetime.utcnow() +
datetime.timedelta(days=0, seconds=60)
        }
        token = jwt.encode(payload, secret_
key, algorithm='HS256')
        session['jwt_token']=token
    except Exception as e:
        return redirect(url_for('home', result="Error 400"))

    return redirect(url_for('home', result="Token Generated
successfully!!"))

@app.route('/validate')
def validate_and_show():
    try:
        token = session.get('jwt_token')
        decode = jwt.decode(token, secret_
key, algorithms=['HS256'])
        return redirect(url_for('home',
result=decode['sub']))
    except:
        return redirect(url_for('home', result="Invalid
token"))

if __name__ == '__main__':
    app.run(debug=True)

```

So what does this code do?

1. First, it imports a JWT package along with a few classes required for our Flask application.
2. Then it declares a secret key, which will be used later in the encoding and decoding of the JWT.

3. `base_url` is the server name. Since I have run it locally, the URL is of `localhost`.
4. After this, it declares three APIs.
 - `home` is used for displaying the home page to the user. The home page is very simple. It contains a welcome message, a link to generate new tokens and a link to *details*. *Details* will be visible to the user only if it is authenticated. A user will have to generate a token first to view the *details*. Any successful attempt messages or invalid token messages will be shown to the user adjacent to the result paragraph.
 - `generate_token` is used for generating a token. JWT has a function `encode` that accepts three parameters — payload, secret key and algorithm -- used for encoding. The payload here is hard coded for explanatory purposes; in real applications we take the user input, and then these inputs become part of the token payload.

The payload is a dictionary containing three values. 'sub' is used to denote the name of the person whose details this token contains. 'iat' denotes the time when the token is created. 'exp' denotes the expiration time for this token. These are registered claims predefined in JWT. So make sure you do not spell these variables.

Here the token will expire within 1 minute after it is generated.

Now this token needs to be stored in the client browser. For that I used `httpOnly` cookies to prevent XSS attacks. Flask provides a configuration value `SESSION_COOKIE_HTTPONLY`, which controls whether cookies are set to be `httpOnly`. By default its value is 'True'. So there is no need to change anything in this step.

I used a session class to store this token in the user browser. I have enclosed all the code in a `try` block. Any exception will display a 400 error to the user; otherwise, it will display a 'Token generated successfully' message.

- `Validate_and_show` is used to validate the token signature and display details only if the signature is verified. When the client requests this API, its cookies will also be sent to the server along with the request. The token is first extracted from the cookies and then decoded using the same `secret_key` used for encoding. The `decode` function takes a different argument as `algorithms`, which is a list of algorithms since the server can expect different types of tokens from the client.

If everything works well, the API will extract the user name from the token and display it to the client. Here, 'bob' will be displayed to the user.



Note: When the token expires or any character of a token string gets changed, the signature will fail and an invalid token message will be displayed to users when they try to look for the details again.